

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: АХО-КОРАСИК

Студентка гр. 4343

Геращенко М.Д.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

1. Цель работы

Цель лабораторной работы - изучить алгоритм Ахо-Корасика и реализовать его для решения задач точного множественного поиска строк, а также для поиска одного шаблона с джокером. В работе также требуется реализовать индивидуальное задание варианта 3: вычислить длину самой длинной цепочки суффиксных ссылок и длину самой длинной цепочки конечных ссылок в построенном автомате.

Для подтверждения корректности программы должны быть подготовлены две версии реализации: чистая версия для автоматической проверки на Stepik и версия с промежуточным выводом для защиты лабораторной работы. В промежуточном выводе должны демонстрироваться построение бора, построение автомата, суффиксные и конечные ссылки, а также процесс поиска по тексту.

2. Постановка задачи

2.1. Задание 1: точный поиск набора образцов

Дан текст T и набор образцов $P = \{p_1, p_2, \dots, p_n\}$. Необходимо найти все вхождения всех образцов в текст. Каждое вхождение выводится в виде пары чисел: позиция начала вхождения в тексте и номер найденного образца.

Нумерация позиций в тексте и нумерация образцов начинаются с единицы.

Строки ответа должны быть отсортированы сначала по возрастанию позиции в тексте, затем по возрастанию номера образца.

2.2. Задание 2: поиск шаблона с джокером

Дан текст T , шаблон P и специальный символ джокера. Джокер совпадает с любым одним символом алфавита. Необходимо найти все позиции вхождения шаблона P в текст T . Один джокер соответствует ровно одному символу, а не подстроке произвольной длины.

В соответствии с уточнением к работе во втором задании реализуется не произвольный алгоритм, а стандартный алгоритм поиска шаблонов с масками на основе алгоритма Ахо-Корасика: шаблон разбивается на безмасочные куски, эти куски ищутся в тексте, после чего с помощью массива счётчиков определяется, в каких позициях входит весь шаблон.

2.3. Индивидуальный вариант 3

Дополнительно к базовой части требуется вычислить две характеристики построенного автомата:

- длину самой длинной цепочки из суффиксных ссылок;
- длину самой длинной цепочки из конечных ссылок.

3. Теоретические сведения

Алгоритм Ахо-Корасика предназначен для одновременного поиска набора строк в тексте. В отличие от последовательного поиска каждого образца отдельно, данный алгоритм строит общий автомат по всем образцам, а затем просматривает текст один раз.

3.1. Бор

Бор - это дерево, в котором каждое ребро подписано символом, а путь от корня до вершины соответствует некоторому префиксу одного или нескольких образцов. Если в вершине заканчивается образец, такая вершина называется терминальной.

Например, для образцов TAGT, TAG и T бор содержит общий путь T -> A -> G -> T. Вершина, соответствующая строке T, терминальная для образца T; вершина TAG - для образца TAG; вершина TAGT - для образца TAGT.

3.2. Суффиксные ссылки

Суффиксная ссылка `link[v]` из вершины `v` ведёт в вершину, соответствующую самому длинному собственному суффиксу строки, задаваемой вершиной `v`. Суффиксные ссылки нужны для того, чтобы при невозможности продолжить текущий путь не начинать поиск заново, а перейти в максимально подходящее состояние автомата.

Например, если вершина соответствует строке `ABCD`, то возможными собственными суффиксами являются `B``CD`, `C``D`, `D` и пустая строка. Суффиксная ссылка ведёт к самому длинному из этих суффиксов, который представлен вершиной в боре.

3.3. Конечные ссылки

Конечная ссылка `terminal_link[v]` ведёт к ближайшей терминальной вершине по цепочке суффиксных ссылок. В литературе такие ссылки также называются терминальными, сжатыми или хорошими суффиксными ссылками. В Викиконспектах для них используется обозначение `up`.

Наличие конечных ссылок является обязательным требованием к данной лабораторной работе. Без конечных ссылок можно получить корректный ответ, проходя по всем суффиксным ссылкам, однако такая реализация может быть менее эффективной: она будет посещать много нетерминальных вершин, в которых не заканчивается ни один образец. Конечные ссылки позволяют переходить только по полезным терминальным состояниям.

3.4. Поиск шаблонов с масками

Для поиска шаблона с джокером используется стандартная схема поиска с масками. Шаблон разбивается на максимальные непрерывные фрагменты, не содержащие джокера. Для каждого такого фрагмента запоминается его смещение внутри исходного шаблона.

Пусть $Q_1, Q_2, \dots, Q_k$ - безмасочные куски шаблона, а $l_1, l_2, \dots, l_k$ - их стартовые позиции в шаблоне. Если кусок Q_i найден в тексте с позиции j , то возможное начало всего шаблона равно $j - l_i$. Для каждой такой позиции увеличивается счётчик S . Если после обработки текста $S[i] = k$, значит все безмасочные куски подтвердили позицию i , и шаблон входит в текст с этой позиции.

4. Описание используемых структур данных

В реализации используется фиксированный алфавит $\{A, C, G, T, N\}$. Каждый символ переводится в номер от 0 до 4, что позволяет хранить переходы в массивах длины 5. Благодаря этому переход по символу выполняется за $O(1)$.

Структура	Назначение
<code>nexts[v][c]</code>	Переходы бора: реальное ребро из вершины v по символу c или -1, если ребра нет.
<code>go[v][c]</code>	Переходы автомата: состояние, в которое автомат перейдёт из вершины v по символу c .
<code>link[v]</code>	Суффиксная ссылка вершины v .
<code>terminal_link[v]</code>	Конечная ссылка: ближайшая терминальная вершина по цепочке суффиксных ссылок.
<code>outputs[v]</code>	Список номеров образцов или блоков, которые заканчиваются в вершине v .
<code>pattern_length</code>	Длины образцов в первом задании.
<code>blocks</code>	Список пар (безмасочный кусок, смещение) во втором задании.
<code>matches_count</code>	Массив S : количество кусков, подтвердивших каждую возможную стартовую позицию шаблона.

5. Алгоритм решения задания 1

1. Считать текст T и количество образцов n .
2. Создать корневую вершину бора.
3. Добавить каждый образец в бор, запоминая номер образца в терминальной вершине.
4. Построить автомат Ахо-Корасика: вычислить переходы `go`, суффиксные ссылки `link` и конечные ссылки `terminal_link`.
5. Пройти по тексту слева направо, поддерживая текущее состояние автомата.
6. После перехода по символу проверить `outputs` текущей вершины.
7. Затем пройти по цепочке `terminal_link` и проверить терминальные вершины, соответствующие суффиксам текущего состояния.
8. Для каждого найденного образца вычислить начальную позицию: $i - \text{len}(\text{pattern}) + 2$, где i - индекс текущего символа в Python.
9. Отсортировать найденные пары по позиции и номеру образца.
10. Вывести ответ.

Сортировка нужна, потому что в процессе обработки одной позиции текста образцы могут добавляться в ответ в порядке, отличном от требуемого условием. Пара (позиция, номер образца) в Python сортируется сначала по первому элементу, затем по второму.

6. Алгоритм решения задания 2

Во втором задании реализован стандартный алгоритм поиска шаблона с масками, описанный для алгоритма Ахо-Корасика. Джокер не добавляется в автомат напрямую, потому что он совпадает с любым символом. Вместо этого шаблон разбивается на точные безмасочные куски.

1. Считать текст T , шаблон P и символ джокера.
2. Разбить шаблон P на максимальные фрагменты, не содержащие джокер.

3. Для каждого фрагмента сохранить пару (строка фрагмента, смещение в шаблоне).
4. Добавить все фрагменты в бор как отдельные образцы.
5. Построить автомат Ахо-Корасика с суффиксными и конечными ссылками.
6. Создать массив `matches_count`, соответствующий массиву `C` из стандартного алгоритма.
7. При нахождении блока в тексте вычислить возможное начало полного шаблона: `block_start_in_text - block_offset`.
8. Если возможное начало не выходит за границы текста, увеличить `matches_count` для этой позиции.
9. После прохода по тексту вывести все позиции `start`, для которых `matches_count[start]` равно количеству блоков.

Например, шаблон `A$$A$` при джокере `$` разбивается на два блока: `A` со смещением 0 и `A` со смещением 3. Если первый блок найден в тексте на позиции 1, а второй на позиции 4, оба блока подтверждают начало полного шаблона в позиции 1.

7. Индивидуальное задание: вариант 3

Для варианта 3 после построения автомата дополнительно вычисляются максимальные длины цепочек ссылок.

7.1. Цепочка суффиксных ссылок

Для каждой вершины `v` рассматривается последовательность `v -> link[v] -> link[link[v]] -> ... -> 0`. Длина такой цепочки равна количеству переходов до корня. Максимум по всем вершинам является длиной самой длинной цепочки суффиксных ссылок.

7.2. Цепочка конечных ссылок

Для каждой вершины v рассматривается последовательность $v \rightarrow \text{terminal_link}[v] \rightarrow \text{terminal_link}[\dots] \rightarrow \dots$ до значения -1. Длина равна количеству реальных переходов по конечным ссылкам. Максимум по всем вершинам является длиной самой длинной цепочки конечных ссылок.

В чистой версии для Stepik эти значения вычисляются, но не выводятся, так как формат ответа строго задан условием. В версии с промежуточным выводом эти значения печатаются в отдельном разделе.

8. Оценка сложности

Пусть $|T|$ - длина текста, S - суммарная длина образцов или безмасочных кусков, $|\Sigma|$ - размер алфавита, A - количество найденных вхождений образцов или кусков.

8.1. Первая задача

- Построение бора: $O(S)$.
- Построение автомата: $O(S * |\Sigma|)$.
- Поиск по тексту с конечными ссылками: $O(|T| + A)$.
- Память: $O(S * |\Sigma|)$.

Так как в работе алфавит фиксирован и имеет размер 5, переходы хранятся в массивах длины 5. Поэтому построение автомата можно считать $O(S)$, а переход по символу выполняется за $O(1)$.

8.2. Вторая задача

- Разбиение шаблона на безмасочные куски: $O(|P|)$.
- Построение бора по кускам: $O(S)$.
- Построение автомата по кускам: $O(S * |\Sigma|)$.
- Поиск кусков в тексте: $O(|T| + A)$.
- Просмотр массива `matches_count`: $O(|T|)$.

Оценка зависит от структуры данных, используемой для хранения переходов автомата. В данной реализации используются массивы фиксированной длины 5, поэтому переход по символу занимает постоянное время. Если бы переходы хранились, например, в словаре или дереве, оценка могла бы измениться в зависимости от стоимости поиска перехода.

9. Вывод

В ходе лабораторной работы был изучен и реализован алгоритм Ахо-Корасика для точного множественного поиска. Были построены бор, автоматные переходы, суффиксные ссылки и конечные ссылки. Наличие конечных ссылок позволяет эффективно находить все образцы, заканчивающиеся в текущей позиции текста, не проходя через лишние нетерминальные вершины.

Для второго задания был реализован стандартный алгоритм поиска шаблона с масками: шаблон разбивается на безмасочные куски, которые ищутся с помощью автомата Ахо-Корасика, а итоговые позиции определяются с помощью массива счётчиков *C*. Индивидуальный вариант 3 выполнен: для построенного автомата вычисляются максимальные длины цепочек суффиксных и конечных ссылок.